



EMBEDDED SOFTWARE ENGINEERING

Session 6 – State Machines

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last time: events, a queue, a timer service – the app reacts, it doesn't poll
- Today: name what's already there. Several places in the project already behave like a state machine, just written as a stray `bool` or an `if`
- Three real examples, from the simplest to the richest – plus a generic one to start, that has nothing to do with the project at all

Today

- 1 What Is a State Machine?
- 2 The Menu's Edit Mode
- 3 Debounce: a State Machine, Entirely in the BSP
- 4 The Time Bomb: Guard Conditions

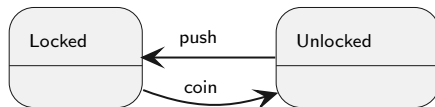
Vocabulary, before any code

- **State:** a situation the system waits in
- **Event:** something that can make it change state
- **Transition:** the move from one state to another, triggered by an event
- **Action:** (optional) something that runs on entering or leaving a state

That's it

No new keyword, no new hardware. A state machine is a switch on "where am I", reacting to "what just happened".

A turnstile



- The classic textbook example, on purpose – nothing about this course's project yet
- 2 states, 2 events, 2 transitions – pushing while locked, or paying while unlocked, is simply ignored (no arrow for it)

The turnstile, in code

```
typedef enum { LOCKED, UNLOCKED } turnstile_state_t;
typedef enum { EVT_COIN, EVT_PUSH } turnstile_event_t;

turnstile_state_t turnstile_transition(turnstile_state_t state,
                                      turnstile_event_t evt) {
    switch (state) {
        case LOCKED:
            return (evt == EVT_COIN) ? UNLOCKED : LOCKED;
        case UNLOCKED:
            return (evt == EVT_PUSH) ? LOCKED : UNLOCKED;
    }
    return state;
}
```

- Reads the current state and the event, returns the next state – nothing more than a switch
- Everything from here on (the menu, debounce, the LEDs) is exactly this mechanic, just with more states or an action attached to entering/leaving one

Already a state machine, just not named

```
typedef struct menu {  
    /* ... */  
    bool editing;  
} menu_t;
```

- *editing* is a 2-valued state – it was already one, since session 5, just not written as one
- Smallest possible step today: give it a name, and dispatch on it explicitly instead of an `if`

Naming it

```
typedef enum {  
    MENU_STATE_BROWSING,  
    MENU_STATE_EDITING,  
} menu_state_t;  
  
typedef struct menu {  
    /* ... */  
    menu_state_t state;  
} menu_t;
```

Dispatching explicitly

```
static bool handle_browsing(menu_t *m, app_t *app, const app_event_t *evt, menu_t **next) {
    /* rotate: move the cursor. click: activate the row, or -> MENU_STATE_EDITING if editable */
}

static bool handle_editing(menu_t *m, app_t *app, const app_event_t *evt, menu_t **next) {
    /* rotate: adjust the value. click: -> MENU_STATE_BROWSING */
}

bool menu_handle_event(menu_t *m, app_t *app, const app_event_t *evt, menu_t **next) {
    *next = NULL;
    switch (m->state) {
        case MENU_STATE_BROWSING: return handle_browsing(m, app, evt, next);
        case MENU_STATE_EDITING:  return handle_editing(m, app, evt, next);
    }
    return false;
}
```

- Same logic as session 5's if (m->editing) {...} else {...} – only the shape changed

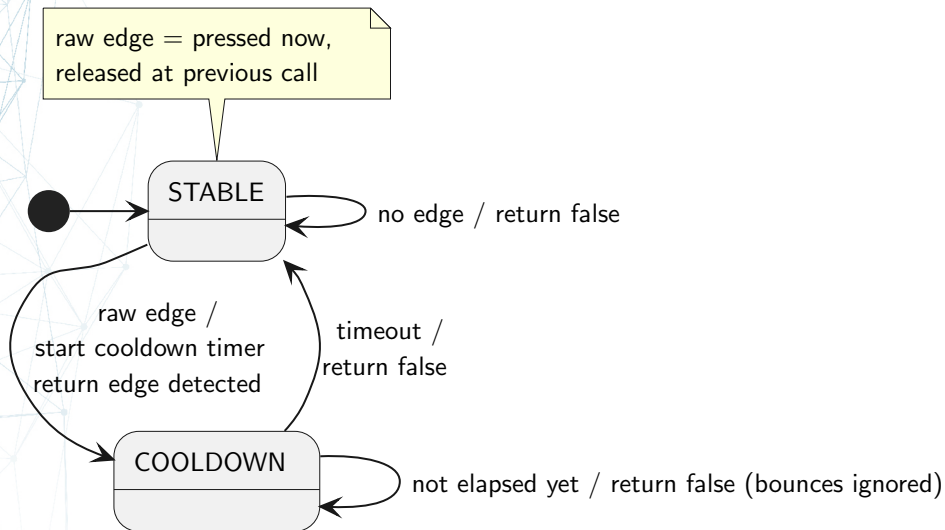
The point of this one

- No entry/exit action needed here – switching state is just writing $m \rightarrow \text{state}$
- So this is nothing more than a transition table, exactly like the turnstile with different names
- The richness comes later – not from this example

The problem

- A mechanical button doesn't produce one clean edge – it bounces, several times, within a few milliseconds
- Without filtering, each bounce becomes its own event – one physical press registers as several clicks
- Good moment to show a state machine doesn't have to live in `app.c`

Two states



debounce_process

```
typedef enum { DEBOUNCE_STABLE, DEBOUNCE_COOLDOWN } debounce_state_t;

typedef struct { debounce_state_t state; uint32_t cooldown_start_ms; } debounce_t;

bool debounce_process(debounce_t *d, bool raw_edge, uint32_t now_ms) {
    switch (d->state) {
        case DEBOUNCE_STABLE:
            if (!raw_edge) { return false; }
            d->state = DEBOUNCE_COOLDOWN;
            d->cooldown_start_ms = now_ms;
            return true;                                /* first edge seen: accepted */
        case DEBOUNCE_COOLDOWN:
            if (now_ms - d->cooldown_start_ms >= DEBOUNCE_PERIOD_MS) {
                d->state = DEBOUNCE_STABLE;
            }
            return false;                                /* never accepted during cooldown */
    }
    return false;
}
```

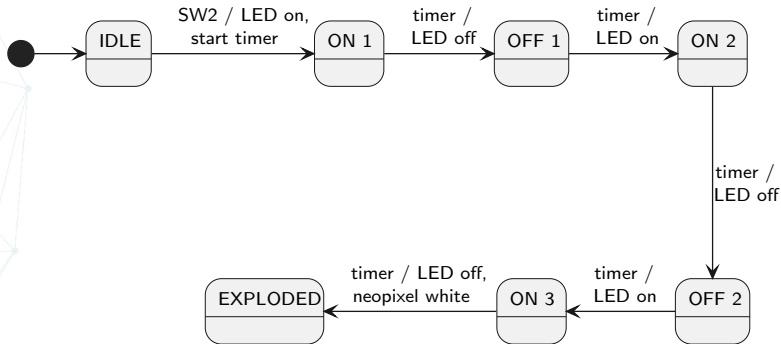
The point of this one

- Doesn't even need the app's software timer service (session 5) – just a raw clock read (`now_ms`), already available to `bsp_process`
- Lives entirely in `bsp.c`, never seen by `app.c` – a state machine is a pattern, not a place

The time bomb: specification

- 1 The *Timebomb* is a new screen, which is not a menu.
- 2 When entering the screen, the system is in a waiting state.
- 3 The timer can be armed by pressing SW2.
- 4 When armed, a countdown starts and blinks an LED
- 5 At the end of the countdown, the board explodes Neopixel goes white.
- 6 It's possible to go back to the menu by pressing SW1.

Blinking three times, without guards



- The count is encoded in the states themselves: one state per step, each timer event moves to the next one
- Works, but 3 blinks need 5 LED states – and 10 blinks would need 19

Guard conditions

- Simple FSM reacts to events
- Sometimes its not enough ; other conditions can be needed
- The *guard* is a boolean condition, evaluated when the event arrives: the transition only fires if it is true
- A transition is written event [guard] / action

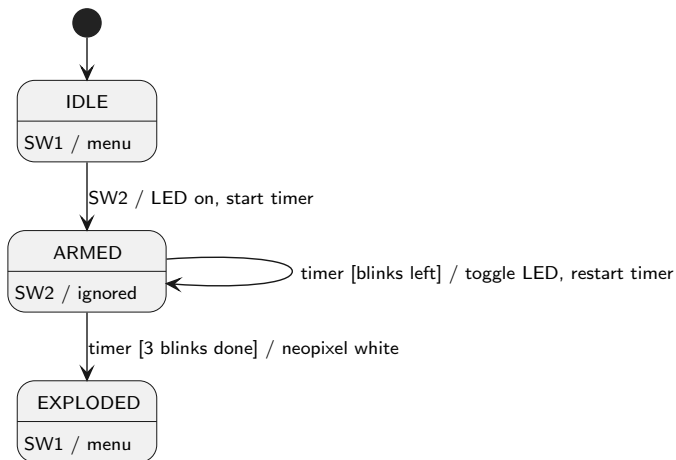
Example: a counter, in state ARMED, receives a timer event

```
case ARMED:
    if (evt->type == EVT_TIMER) {
        if (count < 3) {
            count++;
        } else {
            state = EXPLODED;
        }
    }
    break;
```

/ timer [count < 3] / count++ */*
/ stay in ARMED */*
/ timer [count >= 3] */*
/ go to EXPLODED */*

- Two transitions leave ARMED on the very same event: only the guards tell them apart

The time bomb



Take away

- A state machine is a switch on "where am I", reacting to "what just happened" – nothing more exotic than that
- It doesn't need an enum if a `bool` already covers the state space, and it doesn't need to live in `app.c` – debounce stayed entirely in the BSP
- Actions on entering/leaving a state are what separate "a named variable" from "a machine with real behaviour" – the menu didn't need any, the LEDs did
- Next session: this project, handed over – your turn to add a screen or a state

Exercise: set the countdown

- Goal: in the IDLE state, the encoder sets the number of blinks before the explosion (instead of the fixed `BOMB_BLINK_COUNT = 3`)
- Specification
 - Turning the encoder in IDLE changes the countdown: one step per click of the encoder, between 1 and 10 blinks, no wrap-around
 - The idle screen shows the current countdown
 - SW2 arms the bomb, which then blinks the chosen number of times
 - The encoder is ignored in ARMED and EXPLODED
 - Back in IDLE (re-entering the screen), the countdown goes back to 3
- To do
 - 1 Update the state diagram first: which new transition, with which guard?
 - 2 Then the code: a new field in `time_bomb_t`, a new case in `BOMB_IDLE` of `time_bomb_handle_event` (`EVT_ENCODER_ROTATE`, `evt->data` is the delta), and `time_bomb_draw`
- Think about it: why is the 1–10 limit a guard, and not a state?